

Vehicle Parking App - V2

Project Report T22025

Student Details

Name: Mohammad Akif

Roll Number: DS23F3000281

Subject: Modern Application Development II

1. Project Overview

Problem Statement

Vehicle Parking App - V2 is a comprehensive multi-user application designed to manage different parking lots, parking spots, and parked vehicles specifically for 4-wheeler parking. The system requires two types of users:

- Administrator: Manages parking lots, views analytics, and oversees the entire system
- Regular Users: Books parking spots, views reservations, and manages their parking history

2. Problem Approach & Solution Architecture

System Architecture

The application follows a client-server architecture with clear separation of concerns:

1. Frontend (Vue.js): Single Page Application providing intuitive user interfaces
2. Backend (Flask): RESTful API server handling business logic and data management
3. Database (SQLite): Relational database storing all persistent data
4. Authentication: JWT-based secure authentication system

Key Features Implemented

- Multi-tenant User Management: Role-based access control (Admin/User)
- Real-time Spot Management: Dynamic parking spot allocation and tracking
- Reservation System: Complete booking lifecycle from reservation to release
- Analytics Dashboard: Charts and statistics for parking utilization
- Data Export: CSV export functionality for historical data
- Responsive UI: Modern, mobile-friendly interface design

Development Approach

1. Database Design: Created normalized relational schema
2. API Development: Built RESTful endpoints following REST principles
3. Frontend Development: Implemented reactive UI components
4. Integration: Connected frontend and backend with proper error handling
5. Testing: Manual testing of all user workflows

3. Frameworks and Libraries Used

Backend Technologies:

Python
Flask
Flask-SQLAlchemy
Flask-JWT-Extended
Flask-Login
Flask-Caching
Celery
Werkzeug

Frontend Technologies:

Vue.js
Axios
Bootstrap
FontAwesome
Chart.js

Development Tools:

Vue CLI
Babel
ESLint
npm

4. Database Design - ER Diagram

Entity Relationship Diagram

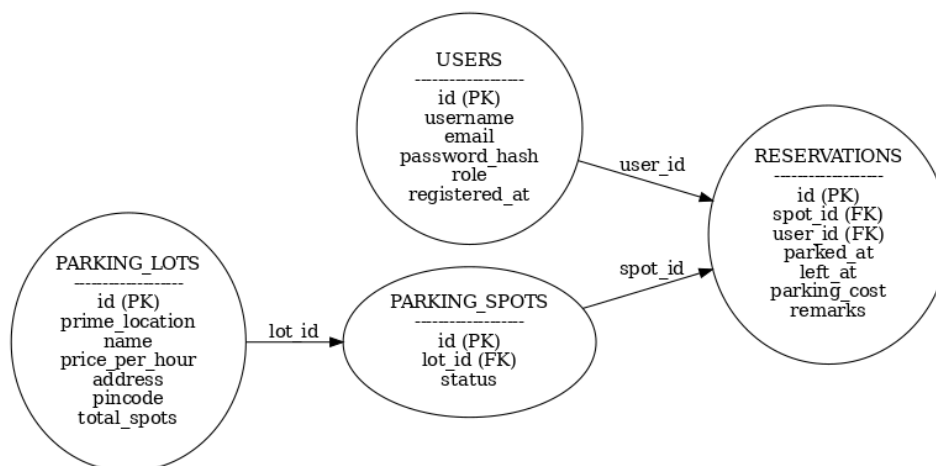


Table Relationships

- Users (1) <-> (N) Reservations: One user can have multiple reservations
- ParkingLots (1) <-> (N) ParkingSpots: One lot contains multiple spots
- ParkingSpots (1) <-> (N) Reservations: One spot can have multiple reservation records
- Users (N) <-> (N) ParkingSpots: Many-to-many relationship through Reservations

Key Constraints

- User email and username must be unique
- Parking spot status: 'A' (Available) or 'O' (Occupied)
- Cascading deletes maintain referential integrity
- Timestamps track reservation lifecycle

5. API Resource Endpoints

Authentication Endpoints

POST /auth/register # User registration
POST /auth/login # User authentication
POST /auth/logout # User logout
GET /auth/me # Get current user profile
PUT /auth/profile # Update user profile

User Endpoints

GET /user/lots # List available parking lots
POST /user/reservations # Create new reservation
GET /user/reservations # Get user's reservations
PUT /user/reservations/<id> # Release parking spot
GET /user/summary # Get user's parking statistics
POST /user/export # Trigger CSV export
GET /user/export/status/<task_id> # Check export status
GET /user/export/download/<filename> # Download CSV file

Admin Endpoints

GET /admin/lots # List all parking lots
POST /admin/lots # Create new parking lot
GET /admin/lots/<id> # Get parking lot details
PUT /admin/lots/<id> # Update parking lot
DELETE /admin/lots/<id> # Delete parking lot
GET /admin/spots/<id> # Get parking spot details
GET /admin/users # List all registered users
GET /admin/search # Advanced search functionality
GET /admin/summary # System-wide analytics

Response Format

All API responses follow consistent JSON format:

```
{  
  "data": { ... },      // Success response  
  "error": "error message", // Error response  
  "message": "status"    // Status message  
}
```

Authentication

- Method: JWT (JSON Web Tokens)
- Header: Authorization: Bearer <token>
- Expiration: Configurable token lifetime
- Role-based Access: Admin vs User permissions

6. Key Features & Functionality

For Users

- Parking Lot Discovery: View available lots with real-time spot counts
- Quick Booking: One-click reservation system
- Reservation Management: Track active and historical bookings
- Cost Calculation: Automatic fare calculation based on duration
- Data Export: Personal parking history in CSV format

For Admin

- Lot Management: Create, update, and delete parking facilities
- Real-time Monitoring: Live view of all parking spots and occupancy
- User Management: View and manage registered users
- Advanced Search: Query system by various criteria
- Analytics Dashboard: Revenue and utilization insights
- Spot Details: Detailed view of individual parking spots

Technical Highlights

- Responsive Design: Works seamlessly on desktop and mobile
- Real-time Updates: Dynamic UI updates without page refresh
- Caching: Optimized performance with intelligent caching
- Error Handling: Comprehensive error messages and validation
- Security: Secure authentication and authorization
- Scalability: Modular architecture for future expansion

7. AI Usage Breakdown

Component	AI Contribution	Human Contribution
System Architecture and Design	20% (recommended fixes and debugging)	80%
Database Schema and Relationships	10% (fixing up relationships)	90%
Flask Backend Development	20% (autocomplete route creation)	80%
Vue.js Frontend Implementation	30% (setting up similar elements in dashboards)	70%
API Design and Implementation	10% (fixing and debugging)	90%
UI/UX Design Concepts	50% (additional functionality to make it look better than barebones)	50%
CSS Styling and Animations	50% (recommendation in colors and styles)	50%
Testing and Integration	5% (test scripts)	95%
Project Documentation	40%	60%

8. Drive Link of Video Presentation

<https://drive.google.com/file/d/1InO8j5WA5fduEoDujcWmdXYj9Od1EXEZ/view?usp=sharing>